```
assert = require 'assert'
{fact} = require './fact'
http = require 'http'

{Tracking} = require './10.4'
{User} = require './10.6'
SERVER_OPTIONS =
  host: 'localhost'
  port: 8080

fact 'the tracking application tracks a user mouse click', ->
  tracking = new Tracking SERVER_OPTIONS, http

  tracking.start ->
    assert.equal tracking.total(), 0
    fred = new User SERVER_OPTIONS, http
    fred.visitPage '/some/url', ->
     fred.clickMouse ->
       assert.equal tracking.total(), 1
       tracking.stop()
```

Server config options—uppercase by convention but not real constants.

Assert the tracking total inside the callback to the asynchronous user mouse click.

The `User` class used in listing 10.5 is shown in the next listing. This class provides a user that can interact inside the tests. Think of it like a test double for a real person. It simulates what happens when a real person visits the site.

Listing 10.6. The `User` class

```
class User
  constructor: (@options, @http) ->
  visitPage: (url, callback) ->
    @options.path = url
    @options.method = 'GET'
    callback()
  clickMouse: (callback) ->
    request = @http.request @options, (request, response) ->
      callback()
    request.end()

exports.User = User
```

Constructor shorthand (chapter 5).

For testing, the user just needs to create the necessary http request.

It’s time to put all of these tests together and create a test suite.

10.6. Test suites